ACTIVIDAD INTEGRADORA

Desarrollo, Validación y Verificación de un Sistema de Software

Propósito

La presente actividad tiene como propósito integrar los conocimientos desarrollados durante la asignatura **Validación y Verificación de Programas**, articulando el desarrollo de un sistema de software con las diferentes técnicas de verificación, validación y testing.
A partir de un problema concreto, los estudiantes deberán **analizar, diseñar y programar una solución**, para posteriormente planificar y ejecutar diferentes tipos de pruebas, detectar y documentar defectos, aplicar técnicas de automatización y elaborar un informe técnico.
La actividad busca aproximar a los estudiantes a una situación similar a la que podrían encontrar en un equipo profesional de desarrollo y testing de software.

1. Modalidad de trabajo

La actividad se realizará en **grupos de 2 integrantes**.

Cada grupo deberá seleccionar una problemática y desarrollar un pequeño sistema de software que permita resolverla.

El sistema podrá ser:

- una aplicación web;
- una aplicación de escritorio;
- una API;
- una aplicación desarrollada en Java, Python, JavaScript u otro lenguaje acordado con el docente.

El sistema deberá contar con **al menos tres funcionalidades principales**.

2. Etapa 1 – Análisis del problema

Antes de comenzar a programar, el grupo deberá presentar una descripción del problema que pretende resolver.

Deberán identificar:

- Problema o necesidad.
- Usuarios del sistema.
- Objetivo.

- Funcionalidades principales.
- Datos de entrada.
- Resultados esperados.
- Restricciones o condiciones.
- Requisitos funcionales.

Deberán elaborar una lista de requisitos que posteriormente permita verificar si el sistema desarrollado cumple con lo solicitado.

3. Etapa 2 – Diseño y planificación

El grupo deberá realizar una planificación inicial del desarrollo y de las pruebas.

Deberán determinar:

- Funcionalidades que serán desarrolladas.
- Integrantes responsables de cada tarea.
- Tecnologías y herramientas.
- Repositorio utilizado.
- Estrategia de pruebas.
- Tipos de pruebas que serán aplicadas.
- Herramientas de testing y automatización.

Se deberá establecer qué aspectos serán verificados durante el desarrollo y cuáles serán validados una vez que el sistema esté funcionando.

4. Etapa 3 – Desarrollo del sistema

El grupo deberá **programar el sistema propuesto**.

El sistema deberá:

- Implementar las funcionalidades definidas.
- Validar los datos de entrada.
- Contemplar situaciones correctas e incorrectas.
- Manejar errores.
- Contar con una estructura de código comprensible.
- Permitir realizar pruebas.

Durante el desarrollo deberán utilizar un sistema de control de versiones, preferentemente **Git**, registrando los avances realizados por los integrantes.

5. Etapa 4 – Verificación del software

Una vez iniciado el desarrollo, el grupo deberá realizar actividades de **verificación**.

Deberán revisar, entre otros aspectos:

- Requisitos.
- Diseño.
- Código.
- Documentación.
- Consistencia entre requisitos y funcionalidades implementadas.

Deberán registrar los problemas encontrados y las acciones realizadas para corregirlos.

Cuando corresponda, podrán realizar:

- Revisiones de código.
- Inspecciones.
- Revisiones cruzadas.
- Análisis estático.
- Revisión de documentación.

6. Etapa 5 – Diseño de casos de prueba

A partir de los requisitos y funcionalidades desarrolladas, deberán diseñar los casos de prueba.
Deberán aplicar las técnicas trabajadas durante la asignatura.
Como mínimo deberán utilizar:

Caja negra

- clases de equivalencia.
- análisis de valores límite.
- datos válidos e inválidos.

Caja blanca

Analizar una parte del código y considerar:

- Instrucciones.
- Decisiones.
- Condiciones.
- Caminos de ejecución.
- cobertura, cuando corresponda.

Cada caso de prueba deberá incluir como mínimo:

Campo	**Descripción**
ID	Identificador del caso

Funcionalidad	Funcionalidad evaluada
Objetivo	Qué se pretende comprobar
Precondiciones	Condiciones necesarias
Datos de entrada	Datos utilizados
Procedimiento	Pasos para ejecutar la prueba
Resultado esperado	Comportamiento esperado
Resultado obtenido	Comportamiento real
Estado	Aprobado / Fallido
Observaciones	Información adicional

7. Etapa 6 – Ejecución de pruebas

Los casos de prueba deberán ejecutarse sobre el sistema desarrollado.

Se deberán realizar pruebas:

- normales;
- con datos inválidos;
- con valores límite;
- con situaciones excepcionales;
- sobre las diferentes funcionalidades.

Los resultados deberán quedar registrados.

Cuando una prueba falle, el grupo deberá analizar si existe un defecto en el sistema y documentarlo.

8. Etapa 7 – Registro y seguimiento de defectos

Cada defecto encontrado deberá registrarse utilizando una plantilla de reporte.

Como mínimo deberá incluir:

- ID del defecto;
- fecha;
- descripción;
- funcionalidad afectada;
- pasos para reproducirlo;
- resultado esperado;
- resultado obtenido;

- evidencia;
- severidad;
- prioridad;
- estado.

El grupo deberá realizar el seguimiento de los defectos hasta su resolución, cuando corresponda.

9. Etapa 8 – Pruebas unitarias

El grupo deberá implementar **pruebas unitarias automatizadas** sobre las principales funciones del sistema.

Se deberán seleccionar al menos **3 funciones o métodos** y crear pruebas que permitan comprobar su funcionamiento.

Si el sistema está desarrollado en Java, se podrá utilizar **JUnit**.

Si se utiliza otro lenguaje, podrá utilizarse una herramienta equivalente.

Deberán presentar:

- código de las pruebas;
- casos evaluados;
- resultados;
- evidencias de ejecución.

10. Etapa 9 – Pruebas de integración

Deberán seleccionar al menos una funcionalidad que involucre la interacción entre dos o más componentes del sistema y realizar una prueba de integración.

Por ejemplo:

Registro de usuario → almacenamiento de datos → inicio de sesión.

Deberán comprobar que los componentes involucrados funcionan correctamente en conjunto.

11. Etapa 10 – Pruebas de sistema y validación

Una vez integrado el sistema, deberán realizar pruebas sobre el sistema completo.

El objetivo será determinar si el producto desarrollado cumple con los requisitos definidos inicialmente.

Deberán seleccionar las funcionalidades principales y comprobar:

- funcionamiento;
- cumplimiento de requisitos;

- manejo de errores;
- comportamiento ante datos incorrectos;
- interacción entre funcionalidades;
- respuesta ante situaciones esperadas y no esperadas.

Finalmente deberán responder:

¿El sistema desarrollado satisface las necesidades planteadas al comienzo del proyecto?

Deberán justificar la respuesta utilizando las evidencias obtenidas durante las pruebas.

12. Etapa 11 – Automatización

El grupo deberá seleccionar pruebas que puedan ser automatizadas.

Deberán automatizar como mínimo **3 casos de prueba**.

Podrán utilizar herramientas como:

- JUnit;
- Selenium;
- Postman;
- PyTest;
- u otra herramienta acordada con el docente.

Deberán explicar:

- herramienta utilizada;
- motivo de su elección;
- pruebas automatizadas;
- procedimiento;
- resultados;
- ventajas y limitaciones encontradas.

13. Etapa 12 – TDD y refactorización

Como parte de la actividad, deberán aplicar el concepto de **Desarrollo Dirigido por Pruebas (TDD)** sobre al menos una funcionalidad.

Deberán mostrar el proceso:

Prueba → implementación → ejecución → corrección/refactorización

Posteriormente deberán realizar una **refactorización** sobre una parte del código, explicando:

- qué problema presentaba el código original;
- qué modificación realizaron;

- por qué la modificación mejora el código;
- qué pruebas utilizaron para comprobar que la funcionalidad continuó funcionando correctamente.

14. Etapa 13 – Pruebas de regresión

Luego de realizar modificaciones o correcciones en el sistema, deberán ejecutar nuevamente un conjunto de pruebas para comprobar que los cambios realizados no afectaron funcionalidades que antes funcionaban correctamente.

Deberán identificar:

- pruebas realizadas antes del cambio;
- modificación realizada;
- pruebas de regresión;
- resultados obtenidos.

15. Etapa 14 – Revisión de artefactos no código

Además del programa, deberán seleccionar al menos un artefacto no relacionado directamente con el código, por ejemplo:

- documentación;
- manual de usuario;
- ayuda del sistema;
- instrucciones de instalación;
- material de capacitación.

Deberán realizar una revisión y registrar los problemas encontrados.

16. Etapa 15 – Informe técnico

El grupo deberá elaborar un **informe técnico final** que documente todo el proceso realizado.

El informe deberá contener:

1. Portada.
2. Integrantes.
3. Descripción del problema.
4. Objetivos.
5. Requisitos.
6. Descripción del sistema.
7. Tecnologías utilizadas.
8. Planificación.
9. Desarrollo.

10. Estrategia de verificación y validación.
11. Casos de prueba.
12. Pruebas de caja negra.
13. Pruebas de caja blanca.
14. Pruebas unitarias.
15. Pruebas de integración.
16. Pruebas de sistema.
17. Automatización.
18. TDD y refactorización.
19. Pruebas de regresión.
20. Defectos encontrados.
21. Seguimiento de defectos.
22. Revisión de artefactos no código.
23. Resultados.
24. Conclusiones.
25. Bibliografía y fuentes consultadas.

El informe deberá incluir las evidencias necesarias para demostrar el trabajo realizado.

17. Presentación y defensa

Cada grupo deberá realizar una presentación final del proyecto.

Durante la exposición deberán mostrar:

- problema planteado;
- sistema desarrollado;
- principales funcionalidades;
- proceso de pruebas;
- casos de prueba;
- defectos encontrados;
- pruebas automatizadas;
- resultados;
- conclusiones.

Además, deberán realizar una **demostración del sistema funcionando**.

Todos los integrantes deberán participar de la presentación.